SHETH LUJ COLLEGE

CYBER SECURITY PRACTICAL - 4

Aim:-Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

CODE:

CS_Practical4.py - D:/daniyal/CS_Practical4.py (3.14.6)

```python
import hashlib
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# Generate RSA private key
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

# Get public key
public_key = private_key.public_key()

# Take message from user
message = input("Enter message: ")

# Convert message into bytes
message_bytes = message.encode()

# Create digital signature using private key
signature = private_key.sign(
    message_bytes,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("\nOriginal Message:", message)
print("Digital Signature:", signature.hex())

# Verify the digital signature
try:
    public_key.verify(
        signature,
        message_bytes,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    print("\nSignature Verification: SUCCESS")
    print("Message is authentic and has not been modified.")
```

CS_Practical4.py - D:/daniyal/CS_Practical4.py (3.14.6)

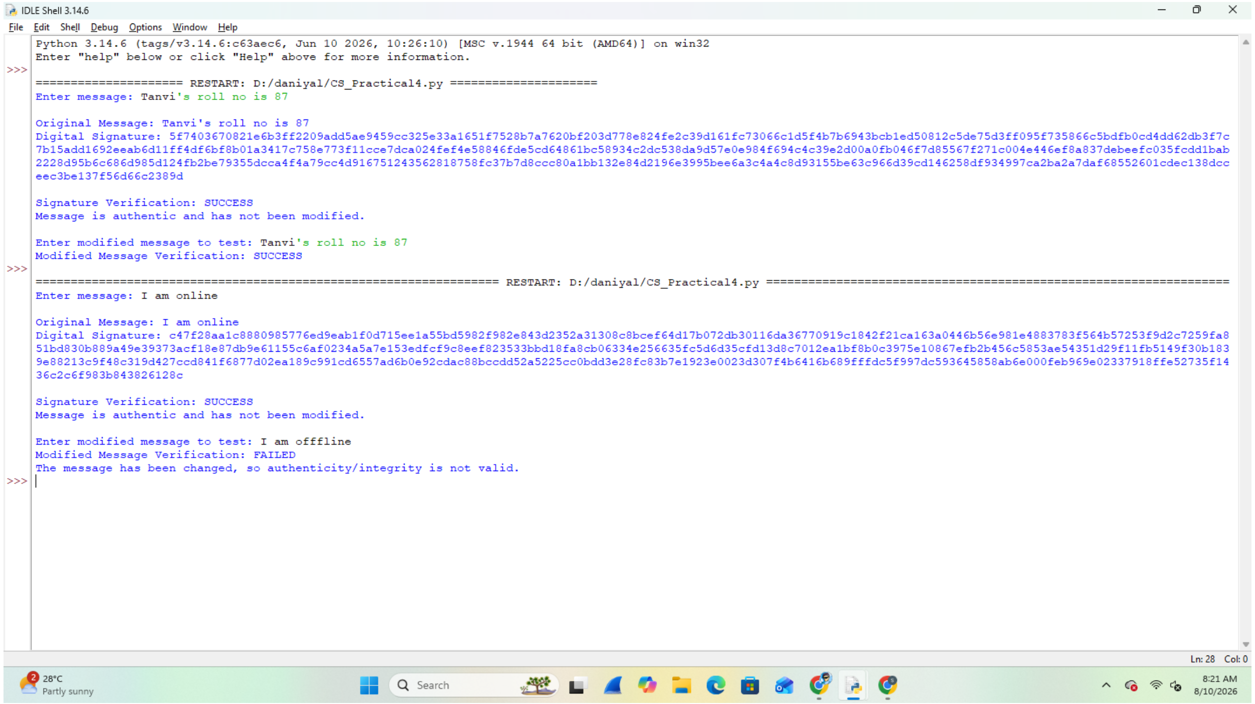
```python
# Convert message into bytes
message_bytes = message.encode()

# Create digital signature using private key
signature = private_key.sign(
    message_bytes,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("\nOriginal Message:", message)
print("Digital Signature:", signature.hex())

# Verify the digital signature
try:
    public_key.verify(
        signature,
        message_bytes,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    print("\nSignature Verification: SUCCESS")
    print("Message is authentic and has not been modified.")

except Exception:
    print("\nSignature Verification: FAILED")
    print("Message has been modified or signature is invalid.")


# Test message integrity
modified_message = input("\nEnter modified message to test: ")

try:
    public_key.verify(
        signature,
        modified_message.encode(),
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
```

OUTPUT:-

TANVI KHAIR
T088

SHETH LUJ COLLEGE

CYBER SECURITY PRACTICAL - 4



TANVI KHAIR
T088